Verilog Operators:

```
+     addition
-     subtraction
*     multiplication
/     division
**    exponent
%     modulus
>     greater than
<     less than
>=    grater than or equal
<=    less than or equal
==    logical equality
!=    logical inequality
===   4-logic value equality
!==   4-logic value inequality
&&    logical and
||    logical or
!     logical negation
&     bit-wise and « a & b  »
&     bit-wize unary and reduction «  &a   »
~&    unary nand reduction
|     bit-wise or « a | b »
|     bit-wize unary or reduction « |a »
~|    reduction nor
^     bit-wise exclusive or « a^b »
^     bit-wize unary or reduction « ^a »
~^    bit-wise equivalence (^~) (also unary reduction
xnor)
~     bit-wise complement
>>    bit-wise logical right shift
<<    bit-wise logical left shift
>>>   bit-wise arithmetic right shift
<<<   bit-wise arithmetic left shift
? :   condition ? value-if-true : value-if-false
*     compare equal on group of bits
=     assignment
<=    non blocking assignment (in clocked processes)
( )   grouping parenthesis, module instantiation,
        function and task call
[ ]   range as in [31:0]
{ }   concatenation { a , 2'b00 , b[2:0] }
{ <n> { } }   repeated concatenation { 16 { 2'b01 } }
```

Literals:

Base:
d = decimal, o = octal, b = binary , h = hexadecimal

Default is decimal: -?[0-9]+
Examples: 0, 1973, -123

Literals with base: [<sign>][<nr_bits>]'<Base><value>
Examples: 'h900D , 'hcafe ,
12'd1239 , -'d12 , 10d123
'b00110100 , 4'b0001

Verilog Types:

4-Value logics of Verilog are defined by: 0,1,x,z

Net Data Types:
wire,
supply0, supply1, tri, triand,
trior, tri0, tri1, wand, wor

Variable Data Types:
reg, integer, real, realtime, time

Vectors
```
wire [31:0] a;
reg [15:0] b;
```

Arrays
```
integer a[0:100];
wire [31:0] a [0:1000];
reg  [31:0] a [0:1000][1:20];
```

SystemVerilog Types

shortint	2-state data type, 16-bit signed integer
int	2-state data type, 32-bit signed integer
longint	2-state data type, 64-bit signed integer
byte	2-state data type, 8-bit signed integer or ASCII character
bit	2-state data type, user-defined vector size, unsigned
logic	4-state data type, user-defined vector size, unsigned
reg	4-state data type, user-defined vector size, unsigned
integer	4-state data type, 32-bit signed integer
time	4-state data type, 64-bit unsigned integer
string	str.len() , str.putc(i,c), str.getc(i) str.itoa(i), str.hextoa(i), str.bintoa(i) str.tolower(), str.toupper(), str.compare() str.substr(i,j)

```
enum {red, yellow, green} light;
typedef [ enum|struct|union|class ] type_identifier;
typedef enum { red, yellow, green } color_t;
```

Control Statements:

```
if(<condition>)
   begin
     <statements>
   end

if(<condition1>)
   begin
     <statements>
   end
  else
   begin
     <statements>
   end
case(<expression>)
   <expression1> : <statement>
   <expression2> : begin
                     <statements>
                   end
   <expression3>,              // note comma
   <expression4>: <statement> // for both
        ...
   default : <statement>      // optional
endcase

for ( var=<init_val> ; <end_cond> ; <var = var + incr>)
  begin
    <statements>
  end

repeat(<count>)
  begin
    <statements>
  end

while(<condition>)
  begin
    <statements>
  end

do
  begin
    <statements>
  end
while(<condition>);

foreach( my_array[idx_var] )
  begin
    <statements using my_array[idx_var]>
  end
```

Delay Statements:

```
`timescale <timeunit>/<precision>;

wait(<condition>) #(<optional_delay) <statement>
wait(<condition>) // waits for condition to become true

@(<some_event>) <statement>
@(<some_event>)
  begin
    <statements>
    end
@(posedge <signal>);
@(negedge <signal>);
#20;  // delay 20 time units
```

Task / Functions:

```
task do_something;
   input  REQ;
   output GNT;
   begin
     // statements. Can consume time
   end
endtask

task do_something(      input int DATA,
                        ouput logic[12:0] RESULT) ;
     // statements. Can consume time
endtask
```

```
function [15:0] max_value;
      input [15:0] a, b;
      begin
        max_value = a < b ? b : a;
      end
 endfunction

 function bit[15:0] max_value(bit[15:0] ,bit[15:0]);
        return a < b ? b : a;
 endfunction
```

Building Blocks:

```
module my_module_name(…);
    parameter <name>=<default value>;

    input <signame>;
    output <signame>;

    assign <wire_name> = <expression>;

    initial  // zero or more initial blocks
      begin
       <sequential statements>
      end

    always  // zero or more always blocks
      begin
       <sequential statements>
      end
    task do_something;
    endtask
    function [15:0] max_value;
    endfunction

    generate ;
       genvar i ;
       for ( i = 0 ; i<10; i++)
       begin : my_name
         <always, initial, instances, signals, interface>
       end
    endgenerate
endmodule
```

Threads

```
forever  // never ending loop
  begin
    <statements>
  end
fork
    <statement>
    <statement>
    ...
join          // wait until all statements complete
// join_none  // wait for none of the threads
// join_any   // wait for any of the thread to complete
```

TESTBENCH

```
module test_adder;
`timescale 1 ns/100 ps
logic [3:0] a, b ; // internal signal declaration
fulladder dut (.a(a), .b(b), .....)); //component instanciation

logic clk = 1'b1;
        always
          #5 clk = ~clk;
        end

initial
 begin
        sig = 0;
 #10; // after 10ns
        sig = 1;
 #10; // 10ns later
        sig = 0;
 // etc. ...
end

endmodule;
```

Blocking vs Non-Blocking Assigments

Blocking : Assignment is performed before moving to next delta cycle

Non-Blocking: Assignment is delayed to the next delta cycle

```
// Non Blocking Assignement
always @(posedge clk)
    begin
      a <= b;
      b <= a;
    end
// Blocking Assignment - Not Synthesieable
always (@posedge clk)
    begin
      tmp = b;
      b = a;
      a = tmp;
    end
// Blocking Assigment - Combinatorial Logic
always @(a or b)
  begin
     if ( a == 12 )
       c = 0 ;
     else
       c = a + b + 15 ;
  end
```

HOW TO DESIGN COMBINATIONAL CIRCUITS

```
module mux3 (input a, b, sel, output f);

always_comb
        begin
                if (sel == 0)
                        f = a;
                else
                        f = b;
        end
endmodule
```

HOW TO DESIGN SEQUENTIAL ELEMENTS

```
module flipflop_syncr (d, q, clk, rst_n);

input d, clk, rst_n;
output logic q;

always_ff @(posedge clk)
        begin
                if (rst_n == 0)
                        q <= 0;
                else
                        q <= d;
        end
endmodule
```

Agregate Data Types

Structures and Unions

```
struct { color_t pix; int a } varA;
varA.pix = yellow;
typedef struct {
        bit[7:0] opcode;
        bit[23:0] addr;
} instruction_s;
instruction_s IR;
IR.opcode = 0'hAE;
typedef union { int i; shortreal f; } num;
```

Packed and Unpacked Arrays

```
bit [7:0] c1;
real u [7:0];
int Array[0:7][0:31];
int Array[8][32];
logic [7:0] mem [0:255];   // declares a memory array
                           // of 256 8-bit elements.
                           // The array indices are 0 to 255
mema[5] = 0;               // Write to word at address 5
data = mema[addr];         // Read word in mem at index = addr
```

Operation on Arrays

Assignments

```
A = B
A[i:j] = B[i:j]
A[x+:c] = B[y+:c]
A[i] = B[i]
```

Comparison

A==B, A[i:j] != B[i:j]

Associative Arrays

```
integer i_array[*];          // associative array of integer
                             // (unspecified index)
bit [20:0] array_b[string];  // associative array
                               // of 21-bit vector, indexed by string
event ev_array[myClass];     // associative array
                             // of event indexed by class myClass

array_b["toto"] = 21'h13cafe;
i_array.delete(12);
int varB = i_array.size();
if ( array_b.exists("titi") )
        array_b.delete("titi");
```

Others:

```
.first(index) , .last(index) , .next(index)
```

Associative Array Methods	Description
.delete(T idx)	Remove the element indexed by idx
.size()	Returns the size of the array
.exists(T idx)	Returns 1 if the list contains an element indexed by idx
.first(T idx)	Returns the first element indexed by idx
.last(T idx)	Relturns the ast element indexed by idx
.next(idx)	Returns the next element of the array